

CVA6 mtvec Readback / Trap-Dispatch Low-Alias Entry Vulnerability Report

Affected version

Issue package: ../../issues/CV-4

This package was validated on a local CVA6 checkout at revision v5.3.0-168-gcd4c8b17, confirmed at commit cd4c8b17c102, using the standard non-H cv64a6_imafdc_sv39 configuration. No fix revision has been identified yet.

Background

The issue package states the implementation bug: software writes a high-tagged mtvec value, reads the same high-tagged value back, and then observes a later M-mode trap dispatch to the low alias. The issue PoC exits with tohost = 42, which is reachable only after mtvec readback preserved the high-tagged value and the subsequent ecall reached the low trap entry.

The security PoC in this package puts that coherence error across a privilege boundary: S-mode prepares bytes in a low-address buffer, while M-mode validates a high-tagged mtvec value by readback before taking a later trap.

Vulnerability type and CWE

Vulnerability type. Machine-mode trap-target validation mismatch and control-flow integrity failure across a privilege boundary.

CWE. Primary: CWE-693 Protection Mechanism Failure. Secondary: CWE-284 Improper Access Control and CWE-670 Always-Incorrect Control Flow Implementation.

Core defect

The core issue is the mismatch between mtvec readback and trap dispatch:

- if CVA6 exposes a high-tagged mtvec value through CSR readback;
- then later M-mode trap dispatch should consume the same architectural base, after any legal CSR-side WARL normalization already reflected in readback.

The issue-level PoC includes an explicit legalization escape hatch:

```
csrw mtvec, s0
csrr s1, mtvec
bne s1, s0, pass_mtvec_legalized
ecall
```

If CVA6 had canonicalized or rejected the high bits at the CSR boundary, the test would have exited 0. Instead, CVA6 preserves the high-tagged value in readback and later enters the low trap entry.

The root cause is the mismatch between the value stored in mtvec_q and the slice consumed by the trap-vector output:

- the CSR_MTVEC write path stores the CSR value using the XLEN slice:

```

riscv::CSR_MTVEC: begin
  logic DirVecOnly;
  DirVecOnly = CVA6Cfg.DirectVecOnly ? 1'b0 : csr_wdata[0];
  mtvec_d = {csr_wdata[CVA6Cfg.XLEN-1:2], 1'b0, DirVecOnly};
  if (DirVecOnly)
    mtvec_d = {csr_wdata[CVA6Cfg.XLEN-1:8], 7'b0, DirVecOnly};
end

```

- the trap-dispatch output path forms the frontend trap base from only the VLEN slice:

```

always_comb begin : priv_output
  trap_vector_base_o = {mtvec_q[CVA6Cfg.VLEN-1:2], 2'b0};

  if (CVA6Cfg.RVS && trap_to_priv_lvl == riscv::PRIV_LVL_S) begin
    trap_vector_base_o =
      (CVA6Cfg.RVH && trap_to_v)
      ? {vstvec_q[CVA6Cfg.VLEN-1:2], 2'b0}
      : {stvec_q[CVA6Cfg.VLEN-1:2], 2'b0};
  end
end

```

Security boundary and realistic scenario

This PoC follows the security model used by the XiangShan XS-4 package: trusted M-mode firmware validates a trap vector by CSR readback before arming it, while less-privileged S-mode software can prepare executable bytes in ordinary low-address memory.

The security boundary is:

1. M-mode starts with a known-safe trap handler.
2. M-mode enters S-mode.
3. S-mode copies a payload into a writable low-address buffer and executes fence.i, making the buffer contain less-privileged prepared code bytes.
4. S-mode returns to M-mode by ecall.
5. M-mode programs mtvec = HIGH_TAG | payload_buf, reads mtvec back, and proceeds only if readback preserved that high-tagged value.
6. M-mode takes another trap. On affected CVA6, trap entry consumes the low alias and executes the S-prepared payload buffer in M-mode.

This is an M-mode control-flow integrity failure. The low-address payload bytes are prepared below M privilege, but a later M-mode trap can enter them after readback-based validation appeared to succeed.

Security PoC

Assumptions

Trusted M-mode firmware treats mtvec readback as validation that the trap target now matches the architectural CSR value. Less-privileged S-mode code can write ordinary memory that is not intended to become an M-mode trap target.

PoC setup

The PoC installs an initial safe M-mode trap handler and opens PMP so the test is about trap-target coherence rather than memory permissions. M-mode enters S-mode at s_prepare_payload.

S-mode copies `payload_template` into `payload_buf`, executes `fence.i`, and uses `ecall` to return to M-mode:

```
s_prepare_payload:
/* copy payload_template to payload_buf */
fence rw, rw
fence.i
ecall
```

The M-mode trap handler for that S-mode `ecall` then validates the high-tagged trap target by readback:

```
la t0, payload_buf
li t1, HIGH_TAG
or t0, t0, t1
csrw mtvec, t0
csrr t2, mtvec
bne t2, t0, pass_mtvec_legalized
ecall
```

The copied payload writes `tohost = 31`. That code exists in the low-address buffer only because S-mode copied it there before M-mode armed the high-tagged `mtvec`.

Security result

- S-mode prepares bytes in `payload_buf`.
- M-mode writes and reads back `HIGH_TAG | payload_buf`.
- M-mode takes a second trap only after the readback check succeeds.
- CVA6 executes the S-prepared payload buffer in M-mode and exits with `tohost = 31`.

CVA6 evidence

The evidence log is `poc/poc.cva6.log`:

```
poc.elf *** FAILED *** (tohost = 31) after 843 cycles
*** [rvfi_tracer] INFO: Simulation terminated after      831 cycles!
```

`tohost = 31` is the PoC's unauthorized payload path. It is reached only after the M-mode trap enters the buffer that was prepared by S-mode.

Expected result on a coherent implementation

There are two acceptable coherent behaviors:

- canonicalize or reject the high-tagged `mtvec` value at the CSR boundary, so readback differs and the PoC exits 0; or
- preserve the high-tagged readback value and make trap dispatch consume that same architectural target, without entering the low-address S-prepared buffer.

In neither case should the PoC execute `payload_buf` and exit with `tohost = 31`.

Observed result on CVA6

CVA6 exits with `tohost = 31`, meaning a tenant-prepared low-address payload buffer became M-mode trap control flow after `mtvec` readback validation appeared to succeed.

Impact

- Primary: Integrity. M-mode trap control flow can enter code bytes prepared by less-privileged S-mode software.
- Secondary: Availability. The same inconsistency can crash, wedge, or redirect trusted firmware by sending M-mode traps into an unintended low alias.

Affected version status

Confirmed affected in a local run of CVA6 revision `v5.3.0-168-gcd4c8b17`, confirmed at commit `cd4c8b17c102`, using the `cv64a6_imafdc_sv39` configuration.

Fixed: none identified yet.

Fix direction

Trap dispatch should consume the same architectural `mtvec` base that software can write and read back. If CVA6 intends to restrict legal trap-target high bits to the implemented virtual or physical address width, it should canonicalize or reject those bits consistently at the CSR boundary instead of exposing one value in `mtvec` and consuming a different effective target at trap entry.